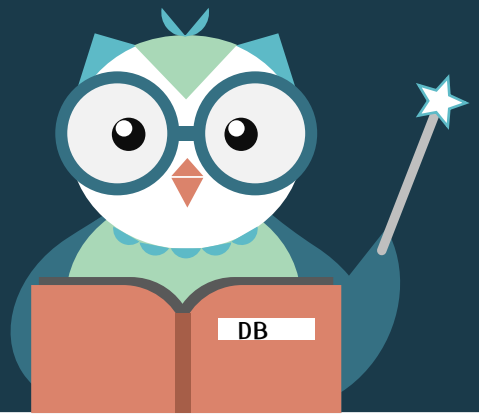
데이터베이스(Database)

박지수



목 차

01 구현 및 테스트 단계

02 테이블 생성·삭제·변경

03 인덱스 생성·삭제

04 초기 데이터 삽입

05 데이터 검색

06 데이터 삽입·삭제·갱신

07 뷰 생성·삭제

08 시퀀스 생성·삭제·변경

09 데이터베이스 구현 사례

학습 목표

- 구현 단계의 주요 업무를 나열할 수 있다.
- 기본 테이블을 생성·삭제·변경할 수 있다.
- 수동으로 인덱스를 생성할 수 있다.
- 기본 테이블에 데이터를 삽입·삭제·갱신할 수 있다.
- 기본키 값을 자동으로 삽입할 시퀀스를 생성할 수 있다.

01 구현 및 테스트 단계

1. 구현 단계의 주요 업무

DB 구현 단계의 주요 업무

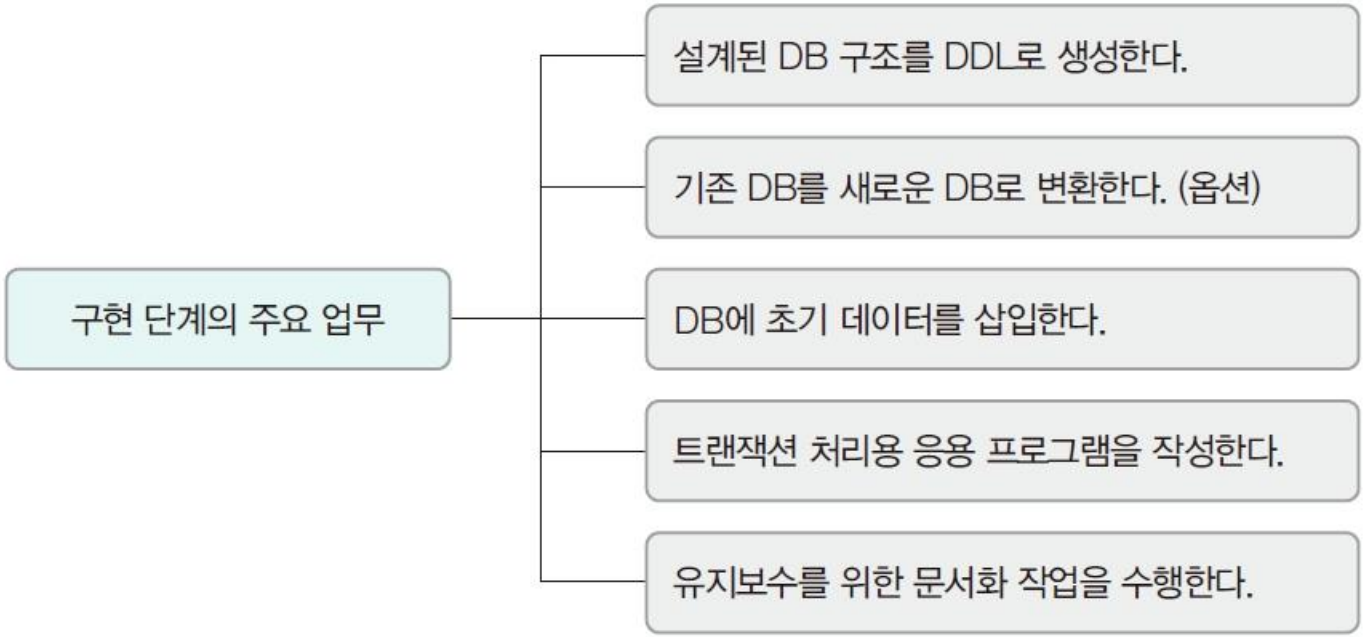


그림 12-1 DB 구현 단계의 주요 업무

2. 테스트 단계의 주요 업무

DB 테스트 단계의 주요 업무

-
-
-

02 테이블 생성·삭제·변경

1. 기본 테이블 생성

테이블(table) 구분

- 기본 테이블(base table) : 독자적으로 존재하는 테이블
- 가상 테이블(virtual table) : CREATE VIEW 명령을 이용해서 기본 테이블로부터 만들어지는 테이블
- 임시 테이블(temporary table) : DDL 문으로 만들어지지 않고, 질의문 처리 과정의 중간 결과로 만들어지는 테이블

1. 기본 테이블 생성

기본 테이블 생성 명령: CREATE TABLE 문

-

- 테이블 생성 권한이 있는 사용자만 사용할 수 있음

```
CREATE  TABLE  기본테이블_이름
   ( {열이름  데이터타입 [NOT NULL] | [DEFAULT 값] | [열_제약조건], }+
     [PRIMARY  KEY  (열이름_리스트),]
     {[UNIQUE  (열이름_리스트),] }*
     {[FOREIGN  KEY(열이름_리스트)
            REFERENCES  참조테이블_이름[(열이름_리스트)]
            [ON DELETE  옵션]
            [ON UPDATE  옵션] ,] }*
     {[CHECK(조건식),] }*
     {[ [CONSTRAINT 제약이름] 테이블_제약조건(열이름_리스트)] }*
);
```

그림 12-2 CREATE TABLE 문의 일반 형식

CREATE TABLE 문의 구성 요소와 기능

표 12-1 CREATE TABLE 문의 구성 요소와 기능

구성 요소	기능
NOT NULL	속성이 널 값을 가질 수 없도록 설정한다.
DEFAULT	속성의 기본값을 설정한다.
열_제약조건	열에 대한 무결성 제약조건 유형(PRIMARY KEY, UNIQUE 등)을 기술한다.
PRIMARY KEY	기본키(개체 무결성 제약조건)를 설정한다.
UNIQUE	속성이 유일한 값을 갖도록 설정한다.
FOREIGN KEY	외래키(참조 무결성 제약조건)를 설정한다. SQL에서는 참조된 테이블의 후보키도 외래키가 될 수 있다.
옵션	NO ACTION, SET NULL, SET DEFAULT, CASCADE 가운데 하나를 설정한다. 예 ON DELETE SET NULL: 피참조 투플이 삭제되면, 널 값으로 갱신되도록 설정한다. 예 ON DELETE CASCADE: 피참조 투플이 삭제되면, 이 투플을 참조하는 투플도 연쇄적으로 삭제되도록 설정한다. 예 ON UPDATE CASCADE: 피참조 투플이 갱신되면, 이 투플을 참조하는 투플도 연쇄적으로 갱신되도록 설정한다.
CHECK(조건식)	삽입 또는 갱신 시에 만족해야 하는 조건을 설정한다.
테이블_제약조건	NOT NULL을 제외한 무결성 제약조건 유형을 작성한다.

대표적인 데이터 타입

표 12-2 대표적인 데이터 타입

데이터 타입	특징
NUMBER(p, s)	소수점을 포함해서 20자리까지의 숫자 기억
INT, INTEGER, SMALLINT	정수 기억
CHAR(n)	고정길이 문자열 기억
VARCHAR(n), VARCHAR2(n)	가변길이 문자열 기억
DATE	YY/MM/DD 형식의 날짜 기억
TIME	hh:mm:ss 형식의 시간 기억
LONG	최대 2기가바이트까지의 가변길이 문자 기억
LOGICAL	논리 값(T/F) 기억

2. 테이블 생성 예제

'사원', '부서', '근무하다'라는 테이블을 생성하는 CREATE TABLE 문

```
CREATE  TABLE  사원
       (사번    INTEGER,
        이름    CHAR(10)   NOT NULL,
        입사일  DATE   NOT NULL,
        호봉    INTEGER,
        휴대폰  CHAR(13),
        자격증  VARCHAR2(100),
        CONSTRAINT  사원_primary_key  PRIMARY KEY(사번),
        CONSTRAINT  사원_휴대폰_unique  UNIQUE(휴대폰));

CREATE  TABLE  부서
       (부서번호 INTEGER  PRIMARY KEY,
        부서명   VARCHAR2(30)  NOT NULL,
        부서장   INTEGER,
        사무실   CHAR(4),
        전화번호 CHAR(12),
        UNIQUE(부서명, 사무실, 전화번호));

CREATE  TABLE  근무하다
       (사원번호  INTEGER,
        부서번호  INTEGER,
        직책      CHAR(20),
        PRIMARY KEY(사원번호, 부서번호),
        FOREIGN KEY(사원번호) REFERENCES 사원(사번)
                              ON DELETE   CASCADE,
        FOREIGN KEY(부서번호) REFERENCES 부서(부서번호)
                              ON DELETE   CASCADE);
```

그림 12-3 사원, 부서, 근무하다 테이블 생성 명령문

2. 테이블 생성 예제

내부 스키마를 기초로 'prof(교수)'와 'dept(학과)' 테이블을 생성하는 CREATE TABLE 문

표 12-3 교수(prof) 릴레이션의 내부 스키마

No.	속성	칼럼 이름	데이터 타입	크기	NULL 허용	키	기타
1	교수번호	prof_id	NUMBER	4		PK	
2	이름	name	VARCHAR2	20	N		
3	전공	major	VARCHAR2	30			
4	학과번호	dept_num	NUMBER	3	N	FK	FK: 학과(학과_id)

표 12-4 학과(dept) 릴레이션의 내부 스키마

No.	속성	칼럼 이름	데이터 타입	크기	NULL 허용	키	기타
1	학과_id	dept_id	NUMBER	3		PK	
2	학과명	dept_name	VARCHAR2	30	N		UNIQUE
3	학과사무실	office	VARCHAR2	10			
4	전화번호	tel	CHAR	12			UNIQUE
5	학과장	chair	NUMBER	4		FK	FK: 교수(교수_id)

2. 테이블 생성 예제

내부 스키마를 기초로 'prof(교수)'와 'dept(학과)' 테이블을 생성하는 CREATE TABLE 문

```
CREATE  TABLE dept (
        dept_id       NUMBER(3),
        dept_name     VARCHAR2(30)  NOT NULL  UNIQUE,
        office        VARCHAR2(10),
        tel           CHAR(12)  UNIQUE,
        chair         NUMBER(4),
        CONSTRAINT    dept_id_pk  PRIMARY KEY(dept_id));

CREATE  TABLE prof (
        prof_id       NUMBER(4),
        name          VARCHAR2(20)  NOT NULL,
        major         VARCHAR2(30),
        dept_num      NUMBER(3)  NOT NULL,
        CONSTRAINT    prof_id_pk  PRIMARY KEY(prof_id),
        CONSTRAINT    dept_num_fk  FOREIGN  KEY(dept_num)
                         REFERENCES  dept(dept_id));

ALTER TABLE dept
ADD CONSTRAINT dept_fk  FOREIGN KEY(chair)
                           REFERENCES prof(prof_id);
```

그림 12-4 dept 및 prof 테이블 생성 명령문

3. 기본 테이블 삭제

기본 테이블 삭제 명령: DROP TABLE 문

-

```
DROP  TABLE  기본테이블_이름;
```

그림 12-5 DROP TABLE 문의 일반 형식

✓ 예) '장비코드'라는 이름의 테이블 삭제하기

```
DROP TABLE  장비코드;
```

3. 기본 테이블 삭제

테이블의 기본키를 다른 테이블이 참조하는 경우

- 먼저 기본키와 함께 기본키를 참조하는 외래키도 함께 삭제하도록 변경한 다음 해당 테이블을 삭제할 수 있음

```
ALTER  TABLE  기본테이블_이름
    DROP  PRIMARY  KEY  CASCADE;
DROP  TABLE  기본테이블_이름;
```

그림 12-6 다른 테이블이 참조하는 기본키를 포함하는 테이블을 삭제하는 명령 형식

✓ 예) 기본키인 dept_id를 prof 테이블의 dept_num이 참조하는 dept 테이블 삭제하기

```
ALTER  TABLE  dept
    DROP  PRIMARY  KEY  CASCADE;
DROP TABLE  dept;
```

4. 기본 테이블 변경

기본 테이블 변경 명령: ALTER TABLE 문

```
ALTER  TABLE  기본테이블_이름
   [ADD  열이름  데이터타입] [DEFAULT 값] |
   [DROP COLUMN  열이름] |
   [MODIFY 열이름  [데이터타입 | DEFAULT 값)] |
   [ADD  CONSTRAINT  무결성제약조건이름  무결성제약조건] |
   [DROP CONSTRAINT  무결성제약조건이름];
```

그림 12-7 ALTER TABLE 문의 일반 형식

ALTER TABLE 문으로 구조를 변경할 수 있는 허용 범위

- 새로운 열을 추가할 수 있다. (ADD 절)
- 기존 열을 삭제할 수 있다. (DROP 절)
- 기존 열의 데이터 타입이나 디폴트 값을 변경할 수 있다. (MODIFY 절)
- 새로운 무결성 제약조건을 추가할 수 있다. (ADD CONSTRAINT 절)
- 기존 무결성 제약조건을 삭제할 수 있다. (DROP CONSTRAINT 절)

4. 기본 테이블 변경

새로운 열 추가하기

```
ALTER  TABLE  사원
         ADD   최종학력   VARCHAR2(20)   DEFAULT   '대졸';
```

기존 열 삭제하기

```
ALTER  TABLE  사원
         DROP   COLUMN  자격증;
```

기존 열의 데이터 타입 변경하기

```
ALTER  TABLE  사원
         MODIFY  이름  VARCHAR2(30);
```

4. 기본 테이블 변경

열의 디폴트 값 변경하기

```
ALTER TABLE 사원
        MODIFY  최종학력  DEFAULT '고졸';
```

무결성 제약조건 삭제하기

```
ALTER TABLE 사원
        DROP CONSTRAINT 사원_휴대폰_unique;
```

무결성 제약조건 추가하기

```
ALTER TABLE 부서
        ADD  CONSTRAINT 부서_foreign_key
                FOREIGN KEY(부서장) REFERENCES 사원(사번);
```

03 인덱스 생성·삭제

1. 인덱스 생성

인덱스 생성 명령: CREATE INDEX 문

-

```
CREATE  INDEX  인덱스이름
          ON     테이블이름(열이름1[, 열이름2, . . .] );
```

그림 12-8 CREATE INDEX 문의 일반 형식

✓ 예) 사원 테이블의 '입사일'과 '호봉' 열에 결합 인덱스 생성하기

```
CREATE  INDEX  사원_입사일호봉_idx
          ON     사원(입사일, 호봉);
```

2. 인덱스 삭제

인덱스 삭제 명령: DROP INDEX 문

-

```
DROP  INDEX  인덱스_이름;
```

그림 12-9 DROP INDEX 문의 일반 형식

- 인덱스 소유자나 DROP ANY INDEX 권한이 있는 사용자만 인덱스를 삭제할 수 있음
- 인덱스는 수정되지 않으므로 수정이 필요한 경우 인덱스를 삭제하고 다시 생성해야 함

✓ 예) 사원_입사일호봉_idx 인덱스 삭제하기

```
DROP  INDEX   사원_입사일호봉_idx;
```

04 초기 데이터 삽입

초기 데이터 삽입 명령: INSERT 문

-

```
INSERT  INTO  테이블_이름 [(열이름_리스트)]
              VALUES (열값_리스트);
```

그림 12-10 INSERT 문의 일반 형식

- VALUES 절에 열의 값을 명시하므로
- 열 이름을 명시하지 않는다면 테이블에 정의된 열의 순서대로 값을 나열해야 함

04 초기 데이터 삽입

✓ 예) 사원, 부서, 근무하다 테이블에 초기 데이터 삽입하기

```
INSERT INTO 사원  VALUES (112014, '김윤수', TO_DATE('04/01/2020','MM/DD/YYYY'), 15,
'010-1111-2222', default);
INSERT INTO 사원  VALUES (112207, '박정미', TO_DATE('09/15/2022','MM/DD/YYYY'), 13,
'010-1234-3322', default);
INSERT INTO 사원  VALUES (122065, '최정훈', TO_DATE('01/10/2020','MM/DD/YYYY'), 12,
'010-1221-5533', '대졸');
INSERT INTO 사원  VALUES (122301, '이정수', TO_DATE('10/10/2023','MM/DD/YYYY'), 10,
'010-7788-6543', '대졸');
INSERT INTO 사원  VALUES (122402, '민효정', TO_DATE('03/15/2024','MM/DD/YYYY'), 10,
'010-1991-9876', '대졸');
INSERT INTO 사원  VALUES (120301, '이정문', TO_DATE('10/10/2024','MM/DD/YYYY'), 10,
'010-7890-1143', default);
INSERT INTO 사원  VALUES (124402, '박미영', TO_DATE('03/15/2024','MM/DD/YYYY'), 10,
'010-2091-0176', default);
INSERT INTO 사원  VALUES (124341, '김민준', TO_DATE('05/15/2024','MM/DD/YYYY'), 10,
'010-4631-1256', default);
INSERT INTO 사원  VALUES (124101, '장문수', TO_DATE('01/02/2024','MM/DD/YYYY'), 11,
'010-3390-3902', '대졸');
INSERT INTO 부서  VALUES (100, '기획부', 112014, 'C105', '041-450-1100');
INSERT INTO 부서  VALUES (200, '개발부', 112207, 'C201', '041-450-1101');
INSERT INTO 부서  VALUES (300, '생산부', 120301, 'C311', '041-450-1102');
INSERT INTO 부서  VALUES (400, '마케팅부', 124402, 'A104', '041-450-1120');
INSERT INTO 부서  VALUES (500, '총무부', 122301, 'A215', '041-450-1121');

INSERT INTO 근무하다  VALUES (112014, 100, '팀장');
INSERT INTO 근무하다  VALUES (112207, 200, '팀장');
INSERT INTO 근무하다  VALUES (122065, 100, '팀원');
INSERT INTO 근무하다  VALUES (122301, 500, '팀장');
INSERT INTO 근무하다  VALUES (122402, 400, '팀원');
INSERT INTO 근무하다  VALUES (120301, 300, '팀장');
INSERT INTO 근무하다  VALUES (124402, 300, '팀원');
INSERT INTO 근무하다  VALUES (124341, 200, '팀장');
INSERT INTO 근무하다  VALUES (124101, 500, '팀원');
```

05 데이터 검색

데이터 검색 명령: SELECT 문

```
SELECT [ALL | DISTINCT] { * | 열이름_리스트  [AS 열별칭] }
FROM       테이블_리스트
[WHERE     조건]
[GROUP BY  열이름_리스트
[HAVING    그룹조건]]
[ORDER BY  열이름_리스트  [ASC | DESC]];
```

그림 12-11 SELECT 명령의 일반 형식

SELECT 문의 구성 요소와 기능

표 12-5 SELECT 문의 구성 요소와 기능

구성 요소	기능
SELECT 절	검색할 하나 이상의 데이터 항목, 즉 열 이름이나 열 별칭(alias)을 기술한다. ① ALL: 검색 결과에서 중복되는 행을 그대로 표시하도록 지정함 ② DISTINCT: 검색 결과에서 중복되는 행을 삭제하도록 지정함 ③ *: 테이블의 모든 열을 검색하도록 지정하는 와일드카드(wildcard) 문자임 ④ AS 열별칭: 열 이름에 새로운 이름을 부여하는 것으로, AS는 생략해도 됨
FROM 절	검색할 테이블 이름을 하나 이상 기술한다.
WHERE 절	검색할 조건을 기술한다.
GROUP BY 절	그룹 질의를 수행할 열 이름을 기술한다. (특정 열의 값이 동일한 행들을 그룹핑해서 그룹별로 한 행의 질의 결과를 생성할 때 사용함)
HAVING 절	GROUP BY 절에 의해 구성된 그룹에 적용할 제한 조건을 기술한다.
ORDER BY 절	특정 열 값을 기준으로 질의 결과를 정렬할 때, 기준이 되는 열 이름을 기술한다. ① ASC: 오름차순(ascending) 정렬을 지정함(기본값임) ② DESC: 내림차순(descending) 정렬을 지정함

05 데이터 검색

모든 열 검색하기

```
SELECT    *
FROM      사원;
```

조건을 만족하는 모든 검색하기

```
SELECT    *
FROM      사원
WHERE     호봉 = 10;
```

조건을 만족하는 특정 열 검색하기

```
SELECT    사번, 입사일
FROM      사원
WHERE     호봉 = 10;
```

중복된 행을 제거하고 검색하기

```
SELECT    DISTINCT  부서번호
FROM      근무하다;
```

행 정렬 검색하기

```
SELECT    사번, 이름, 입사일, 호봉
FROM      사원
ORDER BY  입사일  ASC,  호봉  DESC;
```

GROUP BY 절을 이용하여 검색하기

```
SELECT    입사일, AVG(호봉) AS 평균호봉
FROM      사원
GROUP BY  입사일;
```

HAVING 절을 이용하여 검색하기

```
SELECT    입사일, AVG(호봉) AS 평균호봉
FROM      사원
GROUP BY  입사일
HAVING    AVG(호봉) >= 12;
```

06 데이터 삽입·삭제·갱신

데이터 삽입 명령: INSERT 문

1) 새로운 데이터 직접 삽입하기

- 빈 테이블에 초기 데이터를 삽입하는 방법과 동일

```
INSERT  INTO  테이블_이름 [(열이름_리스트)]
                VALUES (열값_리스트);
```

그림 12-10 INSERT 문의 일반 형식

2) 다른 테이블의 데이터를 복사해서 삽입하기

- 서브쿼리(SELECT_문)를 이용해 검색한 결과를 복사하여 삽입

```
INSERT  INTO  테이블_이름 [(열이름_리스트)]
          SELECT_문;
```

그림 12-12 다른 테이블의 데이터를 복사해서 삽입하는 명령 형식

1. 데이터 삽입

✓ 다른 테이블의 데이터를 복사해서 삽입하는 사례

```
CREATE  TABLE  사원_대졸
( 사번    INTEGER  PRIMARY KEY,
  이름    CHAR(10)   NOT NULL,
  호봉    INTEGER);

INSERT  INTO  사원_대졸(사번, 이름, 호봉)
           SELECT  사번, 이름, 호봉
           FROM    사원
           WHERE  최종학력 = '대졸';
```

2. 데이터 삭제

데이터 삭제 명령: DELETE 문

-

```
DELETE  FROM  테이블_이름
[WHERE  조건];
```

그림 12-13 DELETE 문의 일반 형식

✓ 1개 데이터 삭제 사례

```
DELETE FROM  사원
WHERE  사번 = 124341;
```

데이터 갱신 명령: UPDATE 문

```
UPDATE  테이블_이름
    SET   열이름 = 산술식
    [WHERE  조건];
```

그림 12-14 UPDATE 문의 일반 형식

✓ 1개 데이터 갱신 사례

```
UPDATE  사원
    SET   호봉 = 11
    WHERE  사번 = 122301;
```

✓ 여러 개 데이터 갱신 사례

```
UPDATE  사원
    SET   호봉 = 13
    WHERE  호봉 = 12;
```

07 뷰 생성·삭제

1. 뷰 생성

뷰(view)란?

-

뷰 생성 명령: CREATE VIEW 문

```
CREATE  [FORCE | NOFORCE]
            VIEW  뷰이름  [(열별칭1[,  열별칭2,  . . .]) ]
AS  서브쿼리
[WITH CHECK OPTION [CONSTRAINT  제약이름]]
[WITH READ ONLY];
```

그림 12-15 CREATE VIEW 문의 일반 형식

✓ 뷰가 생성되면 뷰 이름과 뷰 정의가 데이터 사전의 USER_VIEWS 테이블에 저장

CREATE VIEW 문의 구성 요소와 기능

표 12-6 CREATE VIEW 문의 구성 요소와 기능

구성 요소	기능
FORCE	기본 테이블의 존재 여부와 무관하게 뷰를 생성한다.
NOFORCE	기본 테이블이 존재할 때만 뷰를 생성한다.
열별칭	서브쿼리에 의해 선택된 열이나 표현식에 대한 별칭을 지정한다.
서브쿼리	뷰에 포함될 데이터를 검색하는 SELECT 문을 작성한다.
WITH CHECK OPTION	뷰로 접근할 수 있는 행만 삽입 또는 수정되도록 지정한다.
WITH READ ONLY	뷰에 대해서 검색(SELECT) 연산만 가능하고, 다른 DML 연산은 불가능하도록 지정한다.

1. 뷰 생성

단순 뷰 생성하기

```
CREATE  VIEW  사원_2024_view
AS  SELECT  사번, 이름, 입사일, 호봉
    FROM    사원
    WHERE   EXTRACT(YEAR FROM 입사일) = 2024;
```

열 별칭을 사용하여 단순 뷰 생성하기

```
CREATE  VIEW  대졸사원_뷰(id, name, emp_date)
AS  SELECT  사번, 이름, 입사일
    FROM    사원
    WHERE   최종학력 = '대졸';
```

복합 뷰 생성하기

```
CREATE  VIEW  기획부_사원
AS  SELECT  사번, 이름, 입사일
    FROM    사원, 부서, 근무하다
    WHERE   부서명 = '기획부'
    AND     부서.부서번호 = 근무하다.부서번호
    AND     근무하다.사원번호 = 사원.사번;
```

2. 뷰 삭제

뷰 삭제 명령: DROP VIEW 문

```
DROP  VIEW  뷰이름;
```

그림 12-16 DROP VIEW 문의 일반 형식

✓ 앞에서 생성한 '사원_2024_view'를 삭제

```
DROP  VIEW  사원_2024_view;
```

3. 뷰 조작 시 유의 사항

삽입·삭제·갱신이 허용되는 경우

- 뷰가 하나의 기본 테이블에 속한 행과 열의 부분집합으로 정의된 경우

삽입·삭제·갱신이 허용되지 않는 경우

- 뷰의 열이 상수나 산술 연산자 또는 함수가 사용된 산술식으로 만들어진 경우
- 그룹 함수(COUNT, SUM, AVG, MAX, MIN)를 사용해서 뷰가 정의된 경우
- DISTINCT, GROUP BY, HAVING 절을 사용해서 뷰가 정의된 경우
- 2개 이상의 테이블이 관련되어 뷰가 정의된 경우

08 시퀀스 생성·삭제·변경

08 시퀀스 생성·삭제·변경

시퀀스(sequence)란?

-

시퀀스의 특징

- 주로 하는 데 사용됨
-
- 시퀀스 값에 보다 효율적으로 액세스하기 위해 시퀀스 값을 미리 생성해서 메모리에 저장해 두고 사용할 수 있음
- 동일한 시퀀스를

1. 시퀀스 생성

시퀀스 생성 명령: CREATE SEQUENCE 문

```
CREATE  SEQUENCE  시퀀스_이름
                    [INCREMENT  BY  n]
                    [START  WITH  n]
                    [ {MAXVALUE  n | NOMAXVALUE}]
                    [ {MINVALUE  n | NOMINVALUE}]
                    [ {CYCLE | NOCYCLE}]
                    [ {CACHE n | NOCACHE }];
```

그림 12-17 CREATE SEQUENCE 문의 일반 형식

CREATE SEQUENCE 문의 옵션들

표 12-7 CREATE SEQUENCE 문의 옵션

옵션	기능
INCREMENT BY n	시퀀스 번호의 간격을 n(정수)으로 설정한다(생략 시 '1'씩 증가함).
START WITH n	생성할 첫 번째 시퀀스 번호를 n으로 지정한다(생략 시 '1'부터 생성함).
MAXVALUE n	생성할 시퀀스의 최댓값을 n으로 설정한다.
NOMAXVALUE	최댓값의 한계를 정하지 않는다.
MINVALUE n	생성할 시퀀스의 최솟값을 n으로 설정한다.
NOMINVALUE	최솟값의 한계를 정하지 않는다.
CYCLE | NOCYCLE	최댓값 또는 최솟값에 도달한 후, 값을 계속 생성할지 여부를 지정한다(기본 설정 값은 NOCYCLE). CYCLE로 지정한 경우, 최댓값 또는 최솟값에 도달한 후 첫 번째 시퀀스 번호부터 다시 생성한다(단, 기본키에 대해서는 CYCLE 옵션 지정이 불가능함).
CACHE n | NOCACHE	얼마나 많은 시퀀스 값을 미리 생성하여 메모리에 저장할지를 지정한다(n의 기본값은 20임. 즉, 20개의 시퀀스를 미리 생성하여 메모리에 저장해 둠으로써 시퀀스에 액세스하는 효율성을 향상함).

1. 시퀀스 생성

✓ 부서 테이블의 기본키인 부서번호 값을 자동으로 생성하는 데 사용할 시퀀스인 dept_id_seq 생성하기

```
CREATE  SEQUENCE  dept_id_seq
                    INCREMENT  BY  100
                    START  WITH  600
                    MAXVALUE  2000
                    NOCYCLE
                    NOCACHE;
```

2. 의사열 NEXTVAL과 CURRVAL

의사열(pseudo column)이란?

-

- NEXTVAL: 다음에 사용 가능한 시퀀스 값을 생성하는 의사열

  → NEXTVAL에 의해 새 시퀀스 값이 생성된 후 그 값이 CURRVAL에 저장됨
- CURRVAL: 가장 최근에 생성된 현재의 시퀀스 값을 반환하는 의사열

의사열 사용 형식



그림 12-18 NEXTVAL과 CURRVAL 사용 형식

[주의] 의사열을 사용할 때는 CURRVAL을 참조하기 전에 반드시 NEXTVAL을 생성해야 함

2. 의사열 NEXTVAL과 CURRVAL

의사열이 사용 가능한 경우

- 서브쿼리의 일부가 아닌 SELECT 문의 SELECT 절
- INSERT 문에서 서브쿼리의 SELECT 절
- INSERT 문의 VALUES 절
- UPDATE 문의 SET 절

의사열을 사용할 수 없는 경우

- 뷰의 SELECT 절
- DISTINCT 키워드를 사용한 SELECT 문
- GROUP BY, HAVING, ORDER BY를 이용한 SELECT 문
- SELECT, DELETE, UPDATE 문에서의 서브쿼리
- CREATE TABLE, ALTER TABLE 문의 DEFAULT 표현식

3. 시퀀스 사용 사례

✓ 부서 테이블에 '연구부'라는 부서를 하나 삽입하되, 부서번호를 앞에서 생성한 dept_id_seq 라는 시퀀스를 사용하여 삽입하기

```
CREATE  SEQUENCE  dept_id_seq
                  INCREMENT  BY  100
                  START  WITH  600
                  MAXVALUE  2000
                  NOCYCLE
                  NOCACHE;
```

```
INSERT  INTO  부서(부서번호, 부서명)
VALUES  (dept_id_seq.NEXTVAL, '연구부');
```

✓ 현재의 시퀀스 값 확인하기(dummy 테이블인 'dual' 활용)

```
SELECT  dept_id_seq.CURRVAL  FROM  dual;
```

4. 시퀀스 사용 시 유의사항

유의 사항

①

- 시퀀스를 포함한 문장을 롤백하면, 커밋(commit) 이후 롤백 이전에 생성된 시퀀스 번호를 모두 잃게 되므로 이후의 시퀀스에 간격이 생김

✓ 롤백: 모든 미결정된 데이터 변경을 취소하는 것
✓ 커밋: 모든 미결정된 데이터 조작을 영구적으로 변경하는 것

②

- CACHE 옵션을 설정하여 미리 시퀀스를 생성해서 메모리에 저장해 둔 경우, 시스템 손상 등으로 인해 시스템이 비정상적으로 종료되면 미리 생성한 시퀀스 값을 모두 잃어버려 이후의 시퀀스에 간격이 생임

③

- 시퀀스 값이 불규칙적으로 변할 수 있음

5. 시퀀스 변경

시퀀스 변경 명령: ALTER SEQUENCE 문

- 시퀀스의 증가치, 최댓값, 최솟값, 사이클 여부, 캐시 옵션 등 START WITH 옵션을 제외한 나머지 모든 옵션 변경 가능

```
ALTER  SEQUENCE  시퀀스_이름
                    [INCREMENT  BY  n]
                    [ {MAXVALUE  n | NOMAXVALUE}]
                    [ {MINVALUE  n | NOMINVALUE}]
                    [ {CYCLE | NOCYCLE}]
                    [ {CACHE n | NOCACHE }];
```

그림 12-19 ALTER SEQUENCE 문의 일반 형식

✓ 예: 시퀀스 dept_id_seq의 최댓값을 3000으로 변경하기

```
ALTER  SEQUENCE  dept_id_seq
                    MAXVALUE  3000;
```

5. 시퀀스 변경

시퀀스 변경 시 유의 사항

- 시퀀스 생성자나 ALTER SEQUENCE 권한을 가진 사용자만 수정할 수 있다.
- 시퀀스를 수정하면 수정 이후에 생성되는 시퀀스 번호에만 영향을 미친다.
- 시퀀스를 생성할 때 다른 시작 번호부터 다시 생성하려면, 기존 시퀀스를 삭제하고 다시 생성해야 한다.
- 유효성 검사를 자동으로 수행한다.
- 수정된 MAXVALUE가 현재 시퀀스 번호보다 작은 경우, 수정이 허용되지 않는다.

6. 시퀀스 삭제

시퀀스 삭제 명령: DROP SEQUENCE 문

```
DROP  SEQUENCE  시퀀스_이름;
```

그림 12-20 DROP SEQUENCE 문의 일반 형식

✓ 한 번 제거된 시퀀스는 더 이상 참조할 수 없음

09 데이터베이스 구현 사례

온라인 쇼핑몰 DB 생성 및 초기 데이터 삽입

```
CREATE TABLE product (
            prod_no      CHAR(6),
            kind         CHAR(2)   CHECK(kind = '상' OR kind = '하'),
            prod_name    VARCHAR2(50)  NOT NULL,
            prod_size    CHAR(2)  NOT NULL,
            sell_price   NUMBER(7)  NOT NULL,
            buy_price    NUMBER(7)  NOT NULL,
            CONSTRAINT  size_check CHECK(prod_size IN ('대', '중', '소')),
            CONSTRAINT  prod_no_pk  PRIMARY KEY(prod_no));

INSERT INTO  product  VALUES (100001, '상', '티셔츠', '대', 20000, 15000);
INSERT INTO  product  VALUES (100002, '상', '블라우스', '중', 30000, 20000);
INSERT INTO  product  VALUES (100003, '상', '니트', '소', 25000, 18000);
INSERT INTO  product  VALUES (100004, '하', '통바지', '대', 20000, 15000);
INSERT INTO  product  VALUES (100005, '하', '반바지', '중', 25000, 15000);
INSERT INTO  product  VALUES (100006, '하', '카고바지', '소', 30000, 20000);
INSERT INTO  product  VALUES (100007, '상', '자켓', '대', 40000, 30000);
INSERT INTO  product  VALUES (100008, '상', '니트', '대', 30000, 25000);
```

그림 12-21 product(상품) 테이블 생성 및 초기 데이터 삽입 SQL 문

온라인 쇼핑몰 DB 생성 및 초기 데이터 삽입

```
CREATE TABLE customer (
            id          VARCHAR2(20),
            pw          VARCHAR2(20)  NOT NULL,
            name        VARCHAR2(30)  NOT NULL,
            cellphone   CHAR(13)  NOT NULL,
            addr        VARCHAR2(100) NOT NULL,
            point       NUMBER(7)  DEFAULT(1000),
            CONSTRAINT  cell_unique UNIQUE(cellphone),
            CONSTRAINT  customer_id_pk  PRIMARY KEY(id));
INSERT INTO  customer  VALUES ('mhkim', 'kim+1111', '김문호', '010-2233-4455', '천안시 동남구',
DEFAULT);
INSERT INTO  customer  VALUES ('sypark', 'swimswim', '박수영', '010-3423-1234', '서울 종로구',
10000);
INSERT INTO  customer  VALUES ('jangme', 'rose1234', '최장미', '010-9898-5678', '서울 강남구',
DEFAULT);
INSERT INTO  customer  VALUES ('minmin', 'jun2020', '민준민', '010-2039-2930', '천안시 서북구',
20000);
INSERT INTO  customer  VALUES ('mslee', 'soso3344', '이명수', '010-9820-2394', '서울 강서구',
3000);
```

그림 12-22 customer(고객) 테이블 생성 및 초기 데이터 삽입 SQL 문

온라인 쇼핑몰 DB 생성 및 초기 데이터 삽입

```
CREATE TABLE cust_dest_addr (
            cust_id     VARCHAR2(20),
            seq         NUMBER(5),
            addr        VARCHAR2(100),
            CONSTRAINT  cust_dest_pk  PRIMARY KEY(cust_id, seq),
            CONSTRAINT  cust_id_fk  FOREIGN  KEY(cust_id)
                           REFERENCES  customer(id));

INSERT INTO  cust_dest_addr  VALUES ('mhkim', 1, '천안시 동남구');
INSERT INTO  cust_dest_addr  VALUES ('mhkim', 2, '천안시 서북구');
INSERT INTO  cust_dest_addr  VALUES ('sypark', 1, '서울 종로구');
INSERT INTO  cust_dest_addr  VALUES ('sypark', 2, '대전시 중구');
INSERT INTO  cust_dest_addr  VALUES ('jangme', 1, '서울 강남구');
INSERT INTO  cust_dest_addr  VALUES ('mslee', 1, '서울 강서구');
INSERT INTO  cust_dest_addr  VALUES ('mslee', 2, '천안시 서북구');
INSERT INTO  cust_dest_addr  VALUES ('mslee', 3, '부산시 동래구');
```

그림 12-23 cust_dest_addr(고객배송지) 테이블 생성 및 초기 데이터 삽입 SQL 문

온라인 쇼핑몰 DB 생성 및 초기 데이터 삽입

```
CREATE TABLE pur_office (
            shop_no      CHAR(7),
            shop_name    VARCHAR2(50)  NOT NULL,
            tel          VARCHAR2(13)  NOT NULL,
            addr         VARCHAR2(100),
            rep_name     VARCHAR2(30),
            reg_num      CHAR(12)  NOT NULL,
            main_item    VARCHAR2(100),
            CONSTRAINT  pur_off_unique   UNIQUE(tel, reg_num),
            CONSTRAINT  pur_off_pk  PRIMARY KEY(shop_no));
INSERT INTO pur_office VALUES ('mkp1221', '오렌지수트', '02-2347-3920', '서울 강남구', '김철민',
'101-26-23902', '면바지, 면티셔츠');
INSERT INTO pur_office VALUES ('kmn2902', '카라멜봉봉', '02-3901-9876', '서울 강남구', '박영미',
'332-29-76452', '실크블라우스, 니트');
INSERT INTO pur_office VALUES ('opa3287', '나이스샷', '032-4930-6754', '부산시 동래구', '이광호',
'390-23-26743', '스판 바지, 폴리블라우스');
```

그림 12-24 pur_office(구매처) 테이블 생성 및 초기 데이터 삽입 SQL 문

온라인 쇼핑몰 DB 생성 및 초기 데이터 삽입

```
CREATE TABLE sell  (
            sell_num    INTEGER,
            prod_no     CHAR(6),
            cust_id     VARCHAR2(20),
            sell_time   DATE  NOT NULL,
            amount      NUMBER(3)  DEFAULT(1),
            del_day     CHAR(8),
            pay_money   NUMBER(7),
            CONSTRAINT sell_num_pk  PRIMARY KEY(sell_num),
            CONSTRAINT prod_no_fk  FOREIGN  KEY(prod_no)
                        REFERENCES  product(prod_no),
            CONSTRAINT sell_cust_id_fk  FOREIGN  KEY(cust_id)
                        REFERENCES  customer(id));

INSERT INTO  sell  VALUES (1, 100001, 'mhkim', TO_DATE('2023-09-10 16:10',
'YYYY-MM-DD HH24:MI'), 2, '20230912', 40000);
INSERT INTO  sell  VALUES (2, 100002, 'mhkim', TO_DATE('2023-10-15 12:08',
'YYYY-MM-DD HH24:MI'), 3, '20231016', 90000);
INSERT INTO  sell  VALUES (3, 100001, 'sypark', TO_DATE('2023-10-16 18:25',
'YYYY-MM-DD HH24:MI'), 1, '20230912', 20000);
INSERT INTO  sell  VALUES (4, 100002, 'sypark', TO_DATE('2023-10-25 13:15',
'YYYY-MM-DD HH24:MI'), 2, '20231016', 60000);
INSERT INTO  sell  VALUES (5, 100003, 'jangme', TO_DATE('2023-10-26 15:25',
'YYYY-MM-DD HH24:MI'), 1, '20231027', 25000);
INSERT INTO  sell  VALUES (6, 100004, 'mslee', TO_DATE('2023-11-02 09:20',
'YYYY-MM-DD HH24:MI'), 2, '20231102', 40000);
```

그림 12-25 sell(판매하다) 테이블 생성 및 초기 데이터 삽입 SQL 문

온라인 쇼핑몰 DB 생성 및 초기 데이터 삽입

```
CREATE TABLE buy  (
            buy_num     INTEGER,
            prod_no     CHAR(6),
            shop_no     CHAR(7),
            buy_day     CHAR(8)  NOT NULL,
            amount      NUMBER(3),
            buy_money   NUMBER(7),
            CONSTRAINT  buy_num_pk  PRIMARY KEY(buy_num),
            CONSTRAINT  buy_pno_fk  FOREIGN  KEY(prod_no)
                           REFERENCES  product(prod_no),
            CONSTRAINT  buy_sno_fk  FOREIGN  KEY(shop_no)
                           REFERENCES  pur_office(shop_no));

INSERT INTO  buy  VALUES (1, 100001, 'mkp1221', '20230901', 10, 150000);
INSERT INTO  buy  VALUES (2, 100002, 'kmn2902', '20230902', 15, 300000);
INSERT INTO  buy  VALUES (3, 100003, 'kmn2902', '20230901', 10, 180000);
INSERT INTO  buy  VALUES (4, 100004, 'mkp1221', '20230901', 20, 300000);
INSERT INTO  buy  VALUES (5, 100005, 'opa3287', '20230910', 10, 150000);
INSERT INTO  buy  VALUES (6, 100006, 'mkp1221', '20230910', 12, 240000);
INSERT INTO  buy  VALUES (7, 100001, 'mkp1221', '20230910', 15, 150000);
```

그림 12-26 buy(구입하다) 테이블 생성 및 초기 데이터 삽입 SQL 문

온라인 쇼핑몰 DB의 테스트용 SQL 문

```
ALTER TABLE product DROP PRIMARY KEY CASCADE;
DROP TABLE product;
ALTER TABLE customer DROP PRIMARY KEY CASCADE;
DROP TABLE customer ;
DROP TABLE cust_dest_addr;
ALTER TABLE pur_office DROP PRIMARY KEY CASCADE;
DROP TABLE pur_office;
DROP TABLE sell;
DROP TABLE buy;
PURGE RECYCLEBIN;   /* 휴지통 비우기 */
```

그림 12-27 온라인 쇼핑몰 DB의 모든 테이블 및 데이터 삭제 SQL 문

Q & A

jisupark@jj.ac.kr

